

TEST PLAN REVIEWER

AI-Based Test Plan Reviewer

Category:Quality Assurance(QA)

UC ID:QA-13

Submitted by Team18

VELLAM MOUNIKA(23U41A4257)

VALLU VENKATALAKSHMI(23U41A0556)

KONETI TEJA VENKATA ABHISHEK(24U45A0416)

KURAMDASU SOWJANYA(24U45A0227)

Submitted To

INFINITE COMPUTER SOLUTIONS

June 2026

ABSTRACT

Software testing is a critical phase of the Software Development Life Cycle (SDLC), and a well-structured test plan is essential for ensuring software quality. However, test plan reviews are often performed manually, which can lead to inconsistent evaluations, missed test scenarios, inadequate test coverage, and increased review effort. To address these challenges, this project presents TestLens AI, an AI-powered Test Plan Review Platform that automates the review and quality assessment process using the Gemini 2.5 Flash Large Language Model (LLM).

The system provides a modern web-based interface developed using Flask, HTML, CSS, and JavaScript, allowing users to upload or paste test plans for analysis. A predefined Quality Assurance (QA) rubric is used to evaluate the test plan across multiple dimensions, including functional coverage, boundary conditions, negative test cases, risk areas, test data coverage, performance considerations, security considerations, and overall clarity and completeness.

The AI model analyzes the submitted test plan and generates a comprehensive review report containing quality scores, missing test scenarios, coverage gaps, risk assessments, recommendations, and an AI-improved version of the test plan. The results are presented through an interactive dashboard with visual analytics using Chart.js, while review history is maintained using SQLite. Additionally, users can export reports in both PDF and DOCX formats for documentation and future reference.

By providing fast, structured, and consistent feedback, TestLens AI reduces manual effort, improves review accuracy, and helps QA teams identify weaknesses in test plans at an early stage. This project demonstrates the practical application of Artificial Intelligence in Software Quality Assurance and highlights its potential to enhance traditional testing and review processes.

Keywords: Software Testing, Test Plan Review, Artificial Intelligence, Gemini 2.5 Flash, Large Language Model, Quality Assurance, Flask, Software Quality.

INTRODUCTION

In software development, testing plays an important role in ensuring the quality of a product. Before testing begins, a test plan is prepared that describes what needs to be tested and how testing will be performed. However, test plans are often reviewed manually, and the quality of reviews may vary from person to person.

Important test scenarios, risks, or coverage areas can be missed.

To solve this problem, we developed an AI-powered Test Plan Reviewer. This system uses a Large Language Model (LLM) to analyze a test plan and provide feedback based on a predefined QA rubric. The application helps testers identify missing scenarios, risks, coverage gaps, and improvement opportunities.

PROBLEM STATEMENT

Test plans currently receive ad-hoc reviews with inconsistent quality. As a result, important test cases may be overlooked, leading to incomplete testing and software defects.

OBJECTIVE

The main objective of this project is to develop a tool that automatically reviews software test plans and provides useful feedback to improve their quality.

TECHNOLOGIES USED

The following technologies were used in the development of TestLens:

Programming Language - Python

Frontend Technologies - HTML5,CSS3,JavaScript

Backend Framework - Flask

Artificial Intelligence - Gemini 2.5 Flash, Google Generative AI API

Database - SQLite

Data Visualization - Chart.js

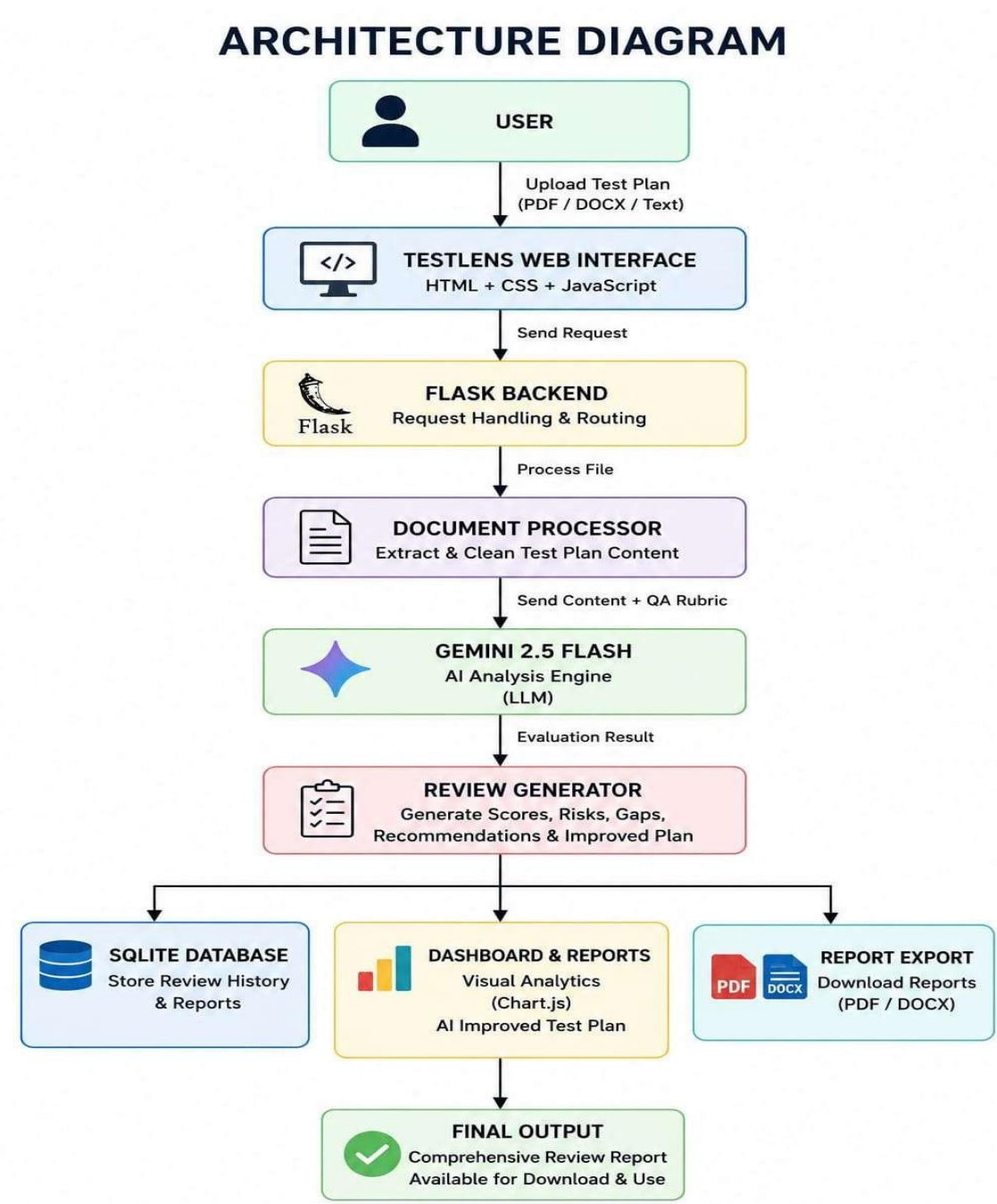
Report Generation - ReportLab (PDF Reports) , Python-Docx (DOCX Reports)

Environment Management - Python-dotenv

Development Tools - Visual Studio Code , Git , GitHub

User Interface Design - Glassmorphism Design, Responsive Web Design

ARCHITECTURE DIAGRAM

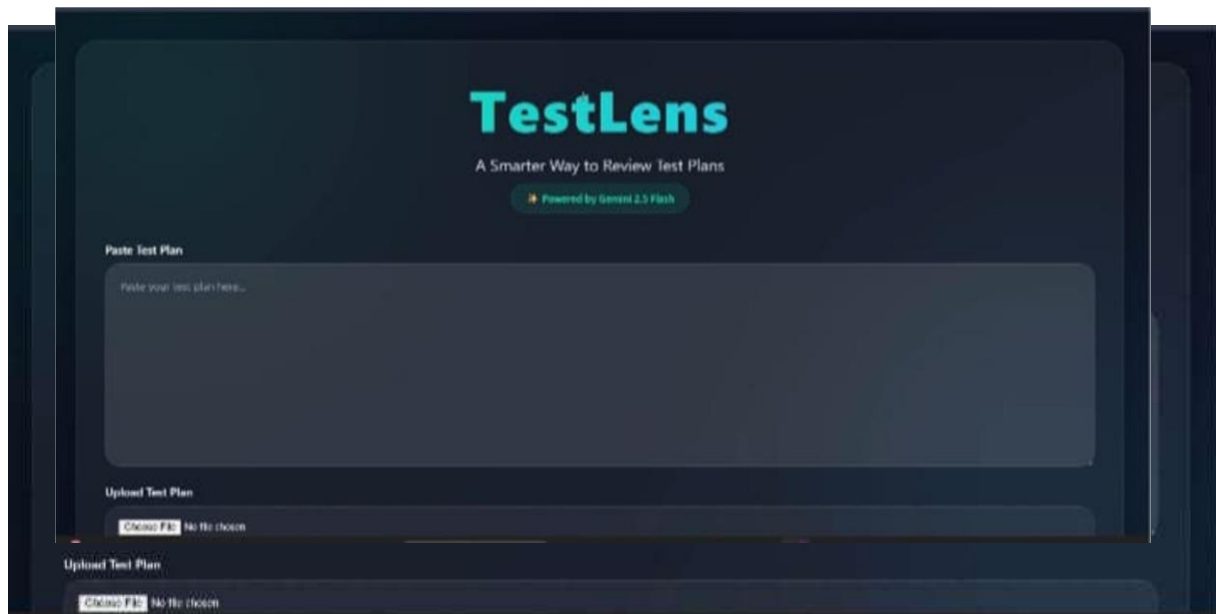


WORKFLOW

The working process of TestLens is simple and efficient:

1. User Submits a Test Plan

The user either pastes a test plan directly into the application or uploads a supported document through the TestLens web interface



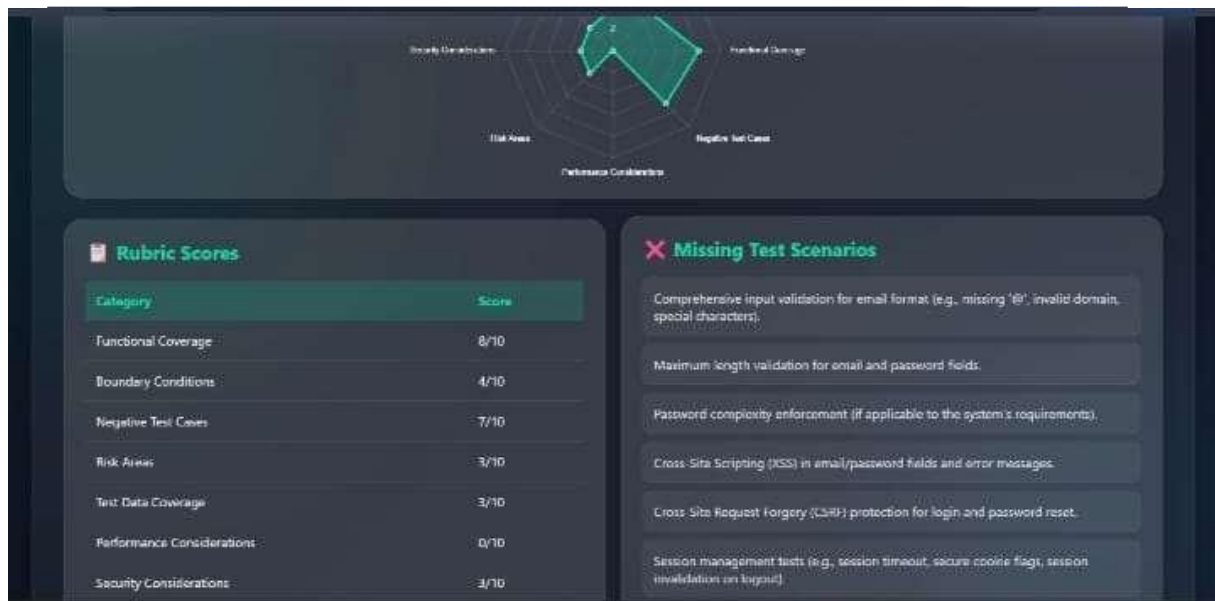
2. Test Plan Processing

The Flask backend receives the input and extracts the test plan content for analysis. Supported file formats are processed and converted into readable text.



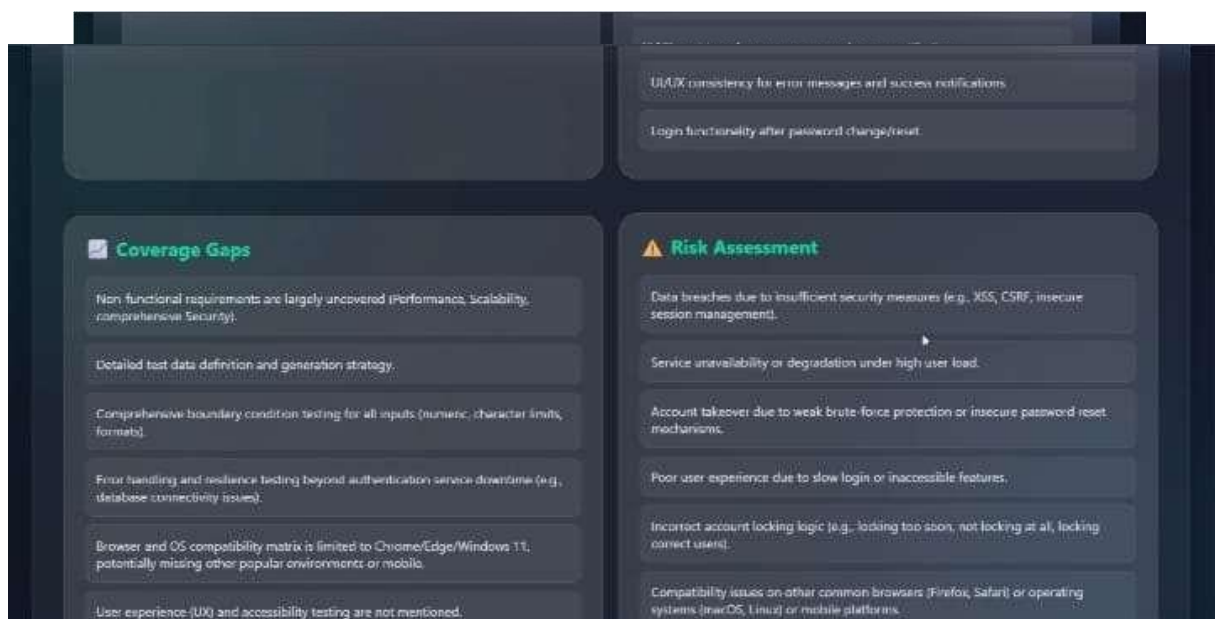
3. QA Rubric Loading

The system loads a predefined Quality Assurance (QA) rubric containing evaluation criteria such as Functional Coverage, Boundary Conditions, Negative Test Cases, Risk Areas, Test Data Coverage, Performance Considerations, Security Considerations, and Clarity & Completeness.



4. AI-Based Analysis

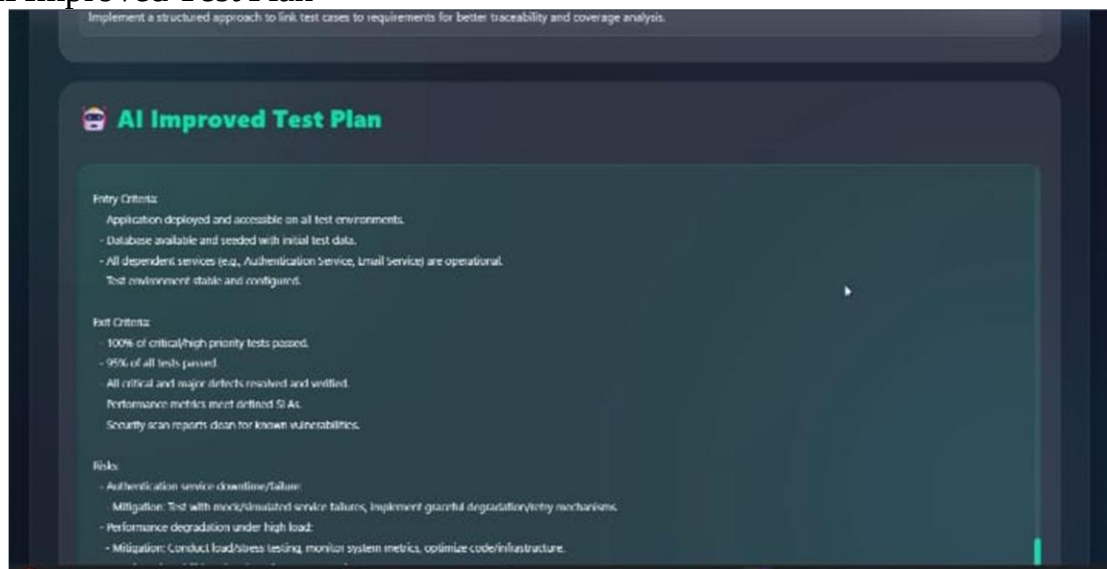
The extracted test plan and QA rubric are sent to the Gemini 2.5 Flash Large Language Model (LLM). The AI analyzes the test plan to evaluate its quality, completeness, and coverage.



5. Review Generation

Based on the analysis, the system generates a structured review report containing:

- Quality Scores
- Coverage Percentage
- Risk Assessment
- Missing Test Scenarios
- Coverage Gaps
- Recommendations
- AI Improved Test Plan



6. Dashboard Visualization

The generated results are displayed on an interactive dashboard with visual analytics, including scorecards and a radar chart for easy interpretation.



7. Review History Storage

All review reports are stored in an SQLite database, allowing users to access previous analyses through the Review History section.

8. Report Export

Users can download the generated review reports in PDF and DOCX formats for documentation, sharing, and future reference.

9. Test Plan Improvement

The user can review the AI-generated feedback and improved test plan to enhance the overall quality, coverage, and effectiveness of the original test plan

SAMPLE INPUT

- Module: Login Module
- Objective:
 - Verify user login functionality.
- Requirements:
 - User should login using valid email and password.
 - Forgot Password functionality should work correctly.
 - Account should be locked after 5 failed login attempts.
- Test Cases:
 1. Valid Login
 2. Invalid Password
 3. Invalid Email
 4. Empty Email Field
 5. Empty Password Field
 6. Password Reset
 7. Account Lock Scenario
- Environment:
 - Chrome
 - Edge
 - Windows 11
- Entry Criteria:
 - Application deployed
 - Database available
- Exit Criteria:
 - 95% test cases passed
- Risks:
 - Authentication service downtime

SAMPLE OUTPUT

- Overall Score: 62/80
- Coverage Percentage: 78%
- Risk Level: Medium
- Rubric Scores
 - Category| Score
 - Functional Coverage| 8/10
 - Boundary Conditions| 6/10
 - Negative Test Cases| 8/10
 - Risk Areas| 7/10
 - Test Data Coverage| 7/10
 - Performance Considerations| 5/10
 - Security Considerations| 8/10
 - Clarity & Completeness| 8/10
- Missing Test Scenarios
 - Session timeout validation
 - Concurrent login attempts
 - Browser refresh handling
 - Password complexity validation
 - Login with special characters
- Coverage Gaps
 - Performance testing scenarios
 - Accessibility validation
 - Cross-browser compatibility coverage
 - Network interruption handling
- Risk Assessment
 - Potential brute-force login attacks
 - Limited performance validation
 - Insufficient boundary testing
- Recommendations
 - Add performance-related test cases

- Include session management testing
 - Expand boundary value testing
 - Add accessibility and usability validation
- AI Improved Test Plan

The system generates an enhanced version of the test plan by incorporating missing scenarios, improving coverage, and strengthening risk validation

CONCLUSION

TestLens is an intelligent test plan review platform designed to improve the quality, consistency, and effectiveness of software test plans. By leveraging the Gemini 2.5 Flash Large Language Model (LLM) and a structured Quality Assurance (QA) rubric, the system automatically evaluates test plans and identifies missing scenarios, coverage gaps, potential risks, and improvement opportunities.

The platform provides detailed review reports, visual analytics, risk assessments, recommendations, and AI-generated improvements, helping QA teams reduce manual review effort and make better testing decisions. Features such as review history tracking, PDF/DOCX report generation, and an interactive dashboard further enhance usability and productivity.

Overall, TestLens demonstrates how Artificial Intelligence can be effectively integrated into Software Quality Assurance processes to improve review accuracy, accelerate feedback cycles, and support the development of higher-quality software systems.

